Sentiment Analysis of Amazon Fine Food Reviews

A Three-Class Sentiment Classifier with Deep Learning

Introduction to Deep Learning Project Report

Instructor: Dr. Trần Việt Trung

Student

Nguyễn Tuệ Lâm 20261207M

November 10, 2026

Contents

1 Introduction **3**
- 1.1 Project Background . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Research Questions . . . 3
- 1.4 Project Objectives . . . 4
- 1.5 Scope and Limitations . . . 4
- 1.6 Report Structure . . . 4

2 Problem Statement and Approach **5**
- 2.1 Review Platforms and Sentiment Monitoring . . . 5
- 2.2 Sentiment from Stars: Weak Labels . . . 5
- 2.3 Business Objectives . . . 5
- 2.4 Evaluation Metrics . . . 6
- 2.5 Success Criteria . . . 6
- 2.6 From Objective to Analytical Problem . . . 6
- 2.7 Approach . . . 6
- 2.8 Implementation Roadmap . . . 7

3 Dataset and Data Specification **8**
- 3.1 Data Source and Collection . . . 8
- 3.2 Dataset Characteristics and Limitations . . . 8
- 3.3 Raw Data Schema . . . 8
- 3.4 Pipeline Column Names . . . 9
- 3.5 Value Conventions . . . 9
- 3.6 Information Not Available . . . 9
- 3.7 Analytical Dataset Design . . . 9

4 Data Understanding **10**
- 4.1 Objective . . . 10
- 4.2 Dataset Overview . . . 10
- 4.3 Class Distribution . . . 10
- 4.4 Text Length . . . 10
- 4.5 Change over Time . . . 11
- 4.6 Users, Products and Helpfulness . . . 11
- 4.7 Noise in the Text . . . 12
- 4.8 Vocabulary of Each Class . . . 12
- 4.9 Design Decisions for Data Preparation . . . 12

5 Data Preparation **13**
- 5.1 Objective . . . 13
- 5.2 Data Cleaning . . . 13
- 5.3 Building the Model Input Text . . . 14

5.4 Label Construction . . . 14
5.5 Train, Validation and Test Split . . . 14
5.6 Class Weights . . . 15
5.7 Leakage Prevention and Random Split . . . 15
5.8 Conclusion . . . 16

6 Modeling **17**
6.1 Objective . . . 17
6.2 Background: Why Sequence Models and Transformers . . . 17
6.3 BiLSTM with Attention . . . 18
6.4 Fine-tuning BERT . . . 20
6.5 Summary of the Models . . . 20

7 Evaluation **21**
7.1 Objective . . . 21
7.2 Results on the Full Test Part . . . 21
7.3 Is the Neutral Class the Problem? . . . 22
7.4 Performance by Review Length . . . 22
7.5 Bootstrap Interval for the Difference . . . 22
7.6 Calibrating the Neutral Threshold . . . 22
7.7 Error Analysis and Examples . . . 23
7.8 Limitations . . . 23

8 Conclusion **25**
8.1 Summary . . . 25
8.2 Limitations . . . 25
8.3 Future Work . . . 25
8.4 Reproducibility . . . 26

Chapter 1

Introduction

1.1 Project Background

Online shops receive thousands of product reviews every day. Reading them by hand is not feasible, yet the sentiment they carry tells a seller which products disappoint buyers and which are praised.

Turning that text into a signal that can be counted is difficult because a star rating and the words of a review do not always agree. A review titled *"Not as Advertised"* that begins "Product arrived labeled as Jumbo Salted Peanuts…the peanuts were actually small sized unsalted" is negative without containing a single strongly negative word. A classifier has to read the sentence, not count words.

This project uses the Amazon Fine Food Reviews corpus [1, 2]: 568,454 reviews posted between October 1999 and October 2012, each with a star rating. The ratings give labels without manual annotation, but they are skewed: after cleaning, about 78% of the reviews are positive and only 7.6% are neutral. A classifier that always answers *positive* reaches about 78% accuracy while recognising nothing about the other classes. Deep learning models that read the text in context are therefore tested on a problem where the obvious shortcut is already strong.

1.2 Problem Statement

- **Difficulty in reading sentiment from text.** Sentiment depends on negation ("not bad"), contrast ("good but…") and context, so counting words is not enough. Models must use word order and context.
- **Imbalanced and approximate labels.** The labels are derived from star ratings, not from human annotation, and the neutral class (3 stars) is rare and ambiguous. A model can look good on accuracy while failing on the minority classes.

1.3 Research Questions

1. How can raw reviews and star ratings be turned into a clean labelled corpus in which the same text never appears in both training and test data?
2. How skewed are the classes and how long are the reviews, and what do these facts imply for the metric, the loss and the sequence length?
3. How much does a pre-trained Transformer fine-tuned on this data (BERT) improve over a recurrent network trained from scratch (BiLSTM with attention)?

4. How far do the two trained models improve on the rule that always answers *positive*, and on which classes?

5. Where do the models fail, in particular on the neutral class and on reviews whose text contradicts their rating?

1.4 Project Objectives

To build and compare deep learning classifiers that assign one of three sentiment labels (negative, neutral, positive) to the text of a food review, using a clean, leakage-safe dataset and an evaluation that is not fooled by the dominant positive class.

1.5 Scope and Limitations

In scope: three-class sentiment classification of English review text, data cleaning and splitting, a BiLSTM with attention trained from scratch, a fine-tuned BERT encoder, and the always-positive rule as reference.

Out of scope / not claimed:

- Human-annotated sentiment – labels come from star ratings, so some are wrong by construction (Section 2.2).
- Predicting the sentiment of *future* reviews – the split is random, not chronological (Section 5.7).
- Aspect-level sentiment (which feature of the product is praised or criticised), other languages and other product domains.
- A deployed service.

1.6 Report Structure

The report is organised into chapters that follow the order in which the work was built. The chapters are:

1. **Introduction** – background, problem statement, research questions, objectives and scope.
2. **Problem Statement and Approach** (Chapter 2) – weak labels, success criteria, metrics and the analytical approach.
3. **Dataset and Data Specification** (Chapter 3) – the Amazon Fine Food Reviews corpus, its schema, conventions and limitations.
4. **Data Understanding** (Chapter 4) – class distribution, text length, change over time, users, noise and vocabulary, and the design decisions they imply.
5. **Data Preparation** (Chapter 5) – cleaning, model input text, stratified split, fixed test subset and class weights.
6. **Modeling** (Chapter 6) – the BiLSTM with attention and the fine-tuned BERT.
7. **Evaluation** (Chapter 7) – test results, confidence intervals, the neutral class, review length, threshold calibration, errors and limitations.
8. **Conclusion** (Chapter 8) – summary, limitations, future work and reproducibility.

Chapter 2

Problem Statement and Approach

2.1 Review Platforms and Sentiment Monitoring

A shop that sells through an online marketplace receives feedback as free text. Each review shows how a customer felt about a product, and a collection of them shows which products are praised and which are criticised. Counting reviews by sentiment turns that text into a number that can be tracked over time and compared between products.

2.2 Sentiment from Stars: Weak Labels

The dataset has no sentiment annotation, only the number of stars (1–5) chosen by the reviewer. The stars are used as a **weak label**: cheap to obtain but only approximately correct. The mapping is fixed once:

Stars (`Score`)	1	2	3	4	5
Label	negative	negative	neutral	positive	positive

A reviewer may give five stars and still write a complaint, so some labels are wrong by construction. This is treated as a property of the data, not hidden, and is examined when the errors of the models are analysed.

2.3 Business Objectives

The objective is to classify the sentiment of a review from its text alone. The specific objectives are:

- **Prepare a trustworthy dataset:** remove repeated reviews before splitting, so the evaluation measures generalisation and not memorisation.
- **Build two neural classifiers:** a BiLSTM with attention trained from scratch, and a BERT encoder fine-tuned on the same data.
- **Compare them fairly:** on identical test reviews, with an always-positive rule as reference.

2.4 Evaluation Metrics

Accuracy is not a sufficient measure here, because always answering *positive* already scores about 78%. For each class c, precision, recall and F1 are

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2\,P_c R_c}{P_c + R_c},$$

and the main metric is **Macro-F1**, the unweighted mean over the three classes:

$$\text{Macro-F1} = \frac{1}{3}\sum_c F1_c.$$

Every class counts equally, so the small neutral class weighs as much as the large positive class. Accuracy, weighted F1, the F1 of each class, the confusion matrix, training time and parameter count are reported as well.

2.5 Success Criteria

Success is defined **relative to simple alternatives** rather than by arbitrary absolute thresholds:

Level	Criterion (evaluated on the unseen test part)
Predictive	Each trained model has a higher Macro-F1 than the always-positive rule (about 0.29).
Transfer learning	The fine-tuned BERT model has a higher Macro-F1 than the BiLSTM trained from scratch, and the paired bootstrap interval of the difference does not contain 0.
Minority classes	The F1 of the negative and neutral classes is reported separately and is above 0 for every model.

2.6 From Objective to Analytical Problem

The input is the English text of one review, made of a short title (`Summary`) and a body (`Text`). The output is one label $y \in \{\text{negative}, \text{neutral}, \text{positive}\}$. The task is to learn a function f_θ from text to $\{0, 1, 2\}$ by minimising a class-weighted cross-entropy loss on the training part (Section 5.6).

2.7 Approach

Raw Reviews (568,454 rows)
↓
Cleaning – remove repeated reviews and invalid rows
↓
Input Text and Label – title + body, label from stars
↓
Stratified Split – train / validation / test, plus a fixed test subset
↓
Training – BiLSTM with attention; BERT fine-tuning; class-weighted loss
↓
Evaluation – Macro-F1, bootstrap intervals, error analysis

2.8 Implementation Roadmap

The approach is implemented as a sequence of steps, each covered by one chapter:

- **Specify the data** (Chapter 3) – source, schema, conventions and what is not available.
- **Understand the data** (Chapter 4) – class balance, length, time, noise, vocabulary; these set the metric, the sequence limits and the cleaning rules.
- **Prepare the data** (Chapter 5) – clean, build the input text, split, and compute class weights.
- **Model and evaluate** – BiLSTM with attention and BERT fine-tuning (Chapter 6) and the comparison of the models (Chapter 7).

Chapter 3

Dataset and Data Specification

3.1 Data Source and Collection

The project uses the publicly available **Amazon Fine Food Reviews** dataset, collected by McAuley and Leskovec [1] and distributed on Kaggle [2]. It is a historical extract of food reviews posted on Amazon between October 1999 and October 2012. The dataset is read from a single CSV file of about 300 MB for inspection, cleaning and modelling.

3.2 Dataset Characteristics and Limitations

- **Weak labels.** The only sentiment signal is the star rating, which does not always agree with the text (Section 2.2).
- **Repeated reviews.** Amazon shares one review among the variants (flavours, sizes) of a product, so the same review appears under several product codes (Section 5.2).
- **One domain.** All reviews concern food products, so results do not necessarily carry over to electronics, books or films.
- **Change over time.** The volume, average rating and share of negative reviews all change between 2003 and 2012 (Section 4.5).

3.3 Raw Data Schema

The raw dataset is a single flat table of 568,454 rows and 10 columns; each row is one review.

Column	Type	Description
`Id`	integer	Row number.
`ProductId`	string	Product code.
`UserId`	string	User code.
`ProfileName`	string	User display name; missing in 26 rows.
`HelpfulnessNumerator`	integer	Number of readers who found the review helpful.
`HelpfulnessDenominator`	integer	Number of readers who voted at all (helpful or not).
`Score`	integer	Star rating from 1 to 5; the source of the label.
`Time`	integer	Posting time as a Unix timestamp.
`Summary`	string	Short review title; missing in 27 rows.
`Text`	string	Review body; complete.

3.4 Pipeline Column Names

The raw columns keep their names in the pipeline. Four columns are derived or combined from them:

Raw column(s)	Pipeline name	Notes
`Score`	`label`	0, 1, 2 for negative, neutral, positive (Section 2.2)
`Score`	`sentiment`	The label as a word, for readable tables and plots
`Time`	`date`	Unix timestamp converted to a date
`Summary`, `Text`	`text`	Model input: cleaned title followed by cleaned body (Section 5.3)

3.5 Value Conventions

Three conventions recur throughout the report:

- **Helpfulness columns.** Amazon shows a line such as "12 of 15 people found this review helpful", which corresponds to `HelpfulnessNumerator` = 12 and `HelpfulnessDenominator` = 15. Because the numerator is a part of the denominator, it can never exceed it. The two columns are used only to check data quality (Section 5.2) and for one exploratory plot (Section 4.6); they are *not* model inputs, because a review that has just been posted has no votes yet.
- **Missing title.** A missing `Summary` is replaced by an empty string; the row is kept.
- **One row, one review.** After cleaning, each row is one review with a unique text. The user and product columns identify authors and items only for the descriptive analysis; they are not model inputs.

3.6 Information Not Available

The dataset contains no human sentiment annotation, no product category, no information on the reviewer beyond an anonymous code and display name, and no predefined train/test split. The split in Chapter 5 is therefore made by this project.

3.7 Analytical Dataset Design

The modelling table has one row per review: the input text, the label, and the original star rating. It is built in Chapter 5, from the cleaned table of Section 5.2.

Chapter 4

Data Understanding

4.1 Objective

Data Understanding and Data Preparation (Chapter 5) form one narrative: understand the data, extract findings about the reviews, and turn them into decisions for building the modelling dataset – its metric, input, split and loss. The business question behind this chapter is which properties of the data must shape the models and their evaluation.

This chapter covers the first half of the narrative: an overview of the data (Section 4.2), class distribution (Section 4.3), text length (Section 4.4), change over time (Section 4.5), users, products and helpfulness (Section 4.6), noise in the text (Section 4.7) and the vocabulary of each class (Section 4.8). It ends with the design decisions (Section 4.9) that Chapter 5 implements. Unless stated otherwise, all figures are measured on the cleaned table of 393,577 reviews produced in Section 5.2.

4.2 Dataset Overview

Metric	Value
Rows in the raw file	568,454
Rows after cleaning	393,577 (69.2% of raw)
Distinct users (cleaned)	256,044
Distinct products (cleaned)	67,554
Date range	October 1999 – October 2012

4.3 Class Distribution

The data lean heavily to the positive side. Positive reviews are roughly 78% of the data, negative 14.5% and neutral 7.6%, so positive outnumbers neutral by about 10.3 to 1. A trivial classifier that always answers *positive* reaches about 78% accuracy yet only 0.29 Macro-F1, because the F1 of the other two classes is zero.

This is why Macro-F1 is the main metric (Section 2.4), and why the split is stratified and the loss is weighted (Chapter 5).

4.4 Text Length

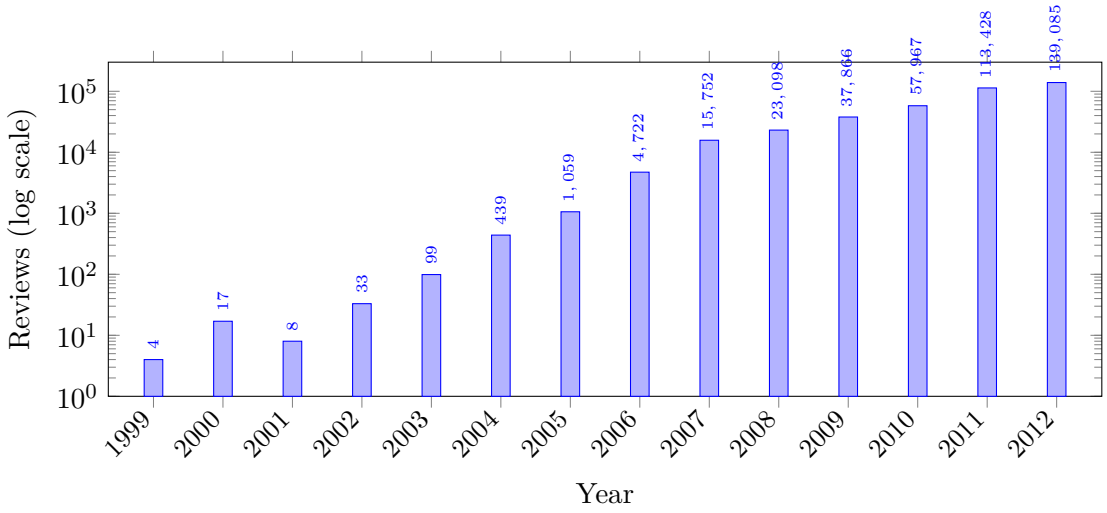
Review length is strongly right-skewed: the median body has 57 words but the longest has 3,432.

	Mean	Std	Min	Median	P90	P95	P99	Max
`Text` (words)	79.8	77.2	3	57	159	215	377	3,432
`Summary` (words)	4.1	2.6	0	3	8	9	12	42

Titles are short (median 3 words) but dense, for example “Great taffy” or “Not as Advertised”, so the title is placed in front of the body to form the model input. Reviews with 2 or 3 stars tend to be longer than those with 5 stars, because the author explains both strengths and weaknesses.

A maximum of 200 words covers about 93% of reviews without truncation, so it is used as the sequence limit for the BiLSTM. For the Transformer the limit is 256 subword tokens; the share of reviews it truncates (about 6.2%) is measured in Section 6.4. When a review is cut, the end is lost and the beginning, which holds the title, is kept. Because lengths differ so much, batches use dynamic padding and length bucketing, so short reviews are not padded to the length of a 3,000-word outlier.

4.5 Change over Time



The number of cleaned reviews grows roughly exponentially from 2006, and 2011–2012 together hold 252,513 reviews (64.2%); 2012 ends in October. The mean star rating drifts down from 4.386 in 2007 to 4.112 in 2012, and the negative share rises from 10.0% to 16.1%. The label distribution is therefore not stable in time.

The split in Section 5.5 is random, so training and test reviews come from the same period. This does not measure how a model would fare on future reviews, and is listed as a limitation.

4.6 Users, Products and Helpfulness

There are 256,044 distinct users and 67,554 distinct products. Of the users, 80.3% wrote a single review, and 78.7% of the products have five reviews or fewer. Both distributions have a long tail, so a model cannot rely on a user’s rating habits and has to learn from the text.

Among reviews with at least five votes, negative reviews attract more votes but a lower helpful ratio, because readers often disagree with a complaint. Since this information is not available when a review is posted, it is not used as a feature (Section 3.5).

4.7 Noise in the Text

Noise in `Text`	Share of reviews
HTML tags (mostly ` `)	26.14%
URLs (`http`, `www`)	2.08%
Encoded HTML characters (`&`, `"`)	0.33%

The text is therefore cleaned lightly (Section 5.3). Stop words and punctuation are *kept*, because negations such as *not*, *no* and *never* appear in standard stop-word lists but decide the sentiment, and because the neural models can learn the role of each word from context.

4.8 Vocabulary of Each Class

Counting words and word pairs on a class-balanced sample of 20,000 reviews per class (stop words removed except negations) shows that single words such as *like*, *taste*, *good* and *coffee* occur in all three classes; *good* is frequent even in negative reviews ("not good", "good but…"). Word pairs separate the classes better: *highly recommend*, *great product* and *love this* in positive reviews; *waste money*, *would not* and *not buy* in negative ones. The neutral class has no vocabulary of its own and mixes that of the other two, so it is expected to be the hardest class.

4.9 Design Decisions for Data Preparation

Finding	Decision
About 31% of raw rows are repeated reviews	Remove them before splitting
Labels skewed 77.9 / 14.5 / 7.6%	Stratified split; class weights in the loss; Macro-F1 as main metric
Right-skewed length; 200 words ≈ 93% coverage	Limit of 200 words (BiLSTM) and 256 tokens (Transformer); dynamic padding and bucketing
Titles are short but informative	Input is title followed by body
26% of bodies contain HTML	Light cleaning; keep punctuation and negations
Context and negation matter	Use a sequence model (BiLSTM with attention) and a Transformer
Label distribution drifts over time	Report as a limitation; the split is random

Chapter 5

Data Preparation

5.1 Objective

Turn the raw reviews into a leakage-safe modelling dataset: clean the raw rows (Section 5.2), build the input text (Section 5.3) and the label (Section 5.4), split the reviews into three parts (Section 5.5) and compute class weights (Section 5.6). The chapter implements the design decisions of Section 4.9 with one frozen configuration:

Parameter	Value	Origin
Split ratio train / val / test	80 / 10 / 10%	Large test part; validation for early stopping
Stratification	By label	Neutral is only 7.6% (Section 4.3)
Random seed	42	Reproducible split
Sequence limit	200 words / 256 tokens	Length coverage (Section 4.4)

5.2 Data Cleaning

Each data-quality issue is first measured on the full raw file of 568,454 rows (rows may overlap across rules):

Issue	Rows	Share of rows
Repeated review (same user, name, time, text)	174,521	30.70%
Repeated `Text`	174,875	30.76%
Missing `Summary`	27	0.005%
Missing `ProfileName`	26	0.005%
Numerator > denominator (helpfulness)	2	<0.001%

No row is duplicated in *all* columns, so the repetition is hidden in the product code. For example, user `A395BORC6FGVXV` posted one 2-star review titled "Cough Medicine" at time 1307923200, and it appears three times under three different product codes (`B000UA0QIQ`, `B002Y7526Y`, `B000U9WZ54`). Left in, such copies would put one on each side of a random split.

The cleaning is performed in this order, always keeping the first occurrence:

1. Drop rows that repeat the key (`UserId`, `ProfileName`, `Time`, `Text`).
2. Drop rows with `HelpfulnessNumerator` > `HelpfulnessDenominator`, which is impossible by definition.

3. Drop rows whose `Text` is still identical to an earlier row's (copies between accounts, spam).

4. Replace a missing `Summary` by an empty string and create the label from `Score` (Section 5.4).

Stage	Rows left	Rows removed
Raw file	568,454	–
After step 1 (repeated key)	393,933	174,521
After step 2 (invalid helpfulness)	393,931	2
After step 3 (repeated `Text`)	393,577	354

Result. Cleaning removes 174,877 rows (30.8% of 568,454) and leaves **393,577 reviews**. Step 3 keeps the first occurrence regardless of its star rating. It also removes legitimate but identical short reviews written by different people, which is a deliberate trade: it costs 354 rows (0.09% of the data) and closes the remaining leakage route. The cleaned table is saved once and read by every later stage.

5.3 Building the Model Input Text

The model input is

$$\text{text} = \text{clean}(\texttt{Summary}) + \text{“. ”} + \text{clean}(\texttt{Text}),$$

where clean unescapes HTML entities, deletes HTML tags and URLs, and collapses whitespace. It does not lowercase, remove punctuation or remove stop words (Section 4.7). The separator follows three rules:

- If the title is non-empty and does not end with “.”, “!” or “?”, the two parts are joined with “. ”.
- If the title already ends with one of those marks, the parts are joined with a single space.
- If the title is empty (27 raw rows have a missing title), the input is the cleaned body alone.

For example, the review titled “Not as Advertised” becomes “Not as Advertised. Product arrived labeled as Jumbo Salted Peanuts…”, and the body fragment “…were bummed.`<br /><br />`Now, because…” becomes “…were bummed. Now, because…”. Cleaning all 393,577 reviews took 19.5 seconds. Tokenisation depends on the model (a word vocabulary for the BiLSTM, WordPiece for the Transformer), so it is done in the modelling chapters.

5.4 Label Construction

The label is derived from `Score` with the mapping of Section 2.2: 1 and 2 stars give negative (0), 3 stars neutral (1), 4 and 5 stars positive (2).

5.5 Train, Validation and Test Split

The data are split in two stratified stages with a fixed seed: 80% for training and 20% held out, then the held-out part is divided equally into validation and test.

```
train_df, temp_df = train_test_split(
    df[cols], test_size=0.2, stratify=df['label'], random_state=SEED)
val_df, test_df = train_test_split(
    temp_df, test_size=0.5, stratify=temp_df['label'], random_state=SEED)
```

Part	Reviews	negative	neutral	positive
Train	314,861	45,654	23,803	245,404
Validation	39,358	same proportions as train		
Test	39,358	same proportions as train		

The proportions are about 14.5%, 7.6% and 77.9% in every part because the split is stratified. Training updates the weights. Validation drives early stopping, the choice of the best epoch and the later calibration of the neutral threshold. Test is used once, at the end, for the reported results. Because the seed and the rule are fixed, a re-run produces the same three parts.

5.6 Class Weights

Let N be the number of training reviews, $K = 3$ the number of classes and N_c the number of training reviews of class c. The weight of class c is

$$w_c = \frac{N}{K\, N_c},$$

and the loss over a batch is

$$\mathcal{L} = -\frac{1}{\sum_i w_{y_i}} \sum_i w_{y_i} \log \hat{p}_{i,y_i}.$$

A rare class receives a large weight, so an error on it costs more and its gradient is not drowned by the majority class. The weights are computed from the training counts only:

Class	Training reviews N_c	Weight w_c
negative	45,654	2.299
neutral	23,803	4.409
positive	245,404	0.428

A neutral review weighs about 10 times as much as a positive one ($4.409/0.428 \approx 10.3$), which is exactly the imbalance ratio of Section 4.3.

5.7 Leakage Prevention and Random Split

- **Repeated text.** Duplicates are removed *before* the split (Section 5.2), so the same text cannot sit in both training and test.
- **Statistics from training only.** Class weights are computed from the training part (Section 5.6); the BiLSTM word vocabulary is built from the training part only (modelling chapter).
- **Test used once.** Epoch choice and the neutral threshold are tuned on validation; test is read for the final report.
- **Random, not chronological.** Training and test reviews come from the same period, although the label distribution drifts (Section 4.5). The scores therefore do not measure performance on future reviews.
- **No automatic check.** The class proportions of each part are printed and compared by eye; there is no assertion, so a change to the split code should be re-checked against the table of Section 5.5.

5.8 Conclusion

The raw 568,454 reviews become 393,577 cleaned reviews, split into 314,861 training, 39,358 validation and 39,358 test reviews, with class weights of 2.299, 4.409 and 0.428 for negative, neutral and positive. The next chapters train the models on the training part, select on validation and report on test.

Chapter 6

Modeling

6.1 Objective

Train two neural networks on the same training part and score them, together with the rule "always answer positive", on the same test reviews (Chapter 5): a BiLSTM with attention trained from scratch (Section 6.3), and a BERT encoder fine-tuned on the data (Section 6.4). The always-positive rule is a reference that is not trained. Section 6.2 first explains what LSTM, BiLSTM, attention and BERT are and why they are used.

Both networks use the class-weighted cross-entropy of Section 5.6, select their settings on the validation part only, and read the test part once. Every number in this chapter comes from the training and validation runs; test results are in Chapter 7.

6.2 Background: Why Sequence Models and Transformers

This section introduces, at the depth the later sections need, the concepts behind the two neural models and the reason each is used here.

Why the order of words matters

A model that only counts words and word pairs cannot tell "not good" from "good" once the negation is two or more words away ("not really all that good"), and it cannot tell who is praised in "the coffee was fine but the delivery was awful". Section 4.8 showed that single words such as *good* occur in all three classes, and that pairs separate the classes better. A model that reads the review *in order* and carries information from earlier words to later ones can use this context. Two families of such models are used.

Recurrent networks and the LSTM

A recurrent network reads the words one at a time and keeps a hidden state h_t that summarises everything read so far. A plain recurrent network loses information over long texts, because the gradient that trains it shrinks at every step (the *vanishing gradient* problem). The **Long Short-Term Memory** (LSTM) unit fixes this with a memory cell c_t and three gates, which decide what to forget, what to write and what to output:

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), & i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i),\\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o), & c_t &= f_t \odot c_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t] + b_c),\\
h_t &= o_t \odot \tanh(c_t), &&
\end{aligned}
$$

where x_t is the vector of the t-th word and σ is the logistic function. The cell c_t is changed only by addition and element-wise gating, so information, and gradient, can travel across many words. An LSTM therefore suits reviews of tens to hundreds of words.

Why bidirectional, and why attention

A one-directional LSTM reads left to right, so its state at a word knows only the words before it. A **bidirectional LSTM** (BiLSTM) runs a second LSTM from right to left and joins the two states, $h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}]$. The state at each word then depends on the words on both sides, which helps when the meaning of a word is set by what follows it ("*good … but not worth the price*").

The sentiment of a review is not carried equally by every word. **Attention pooling** learns a weight α_t for each position and forms the review vector as the weighted sum of the states (equations in Section 6.3), instead of keeping only the last state, where early words are weakest. The weights can also be shown on the text, which makes the model inspectable.

BiLSTM with attention was chosen because it is a standard, small sequence model for sentiment that can be trained from scratch on this data in minutes, it reads context in both directions, and it is inspectable.

The Transformer and BERT

A recurrent network passes information from word to word, so a distant pair of words is linked only through many steps. The **Transformer** replaces this with *self-attention*: every token compares itself directly with every other token of the review,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where the queries Q, keys K and values V are linear maps of the token vectors. Several such *heads* work in parallel, and a vector for the position of each token is added because self-attention alone ignores order. A stack of such layers is an *encoder*.

BERT (Bidirectional Encoder Representations from Transformers) is a Transformer encoder that was first *pre-trained*, by its authors, on a very large amount of English text (Wikipedia and BookCorpus) without any sentiment labels. Its task was to predict words hidden in the text (masked language modelling) and whether one sentence follows another. This teaches the network the grammar, the meaning of words in context and common negation patterns. Text is split into WordPiece sub-words, so a rare word becomes known pieces (*unbelievable* → *un*, *##bel*, *##ie*, …) and no word is unknown. The first token of every input is a special `[CLS]` token whose final vector represents the whole text.

Fine-tuning is *transfer learning*: a small output layer for the three classes is added on top of the `[CLS]` vector, and the whole pre-trained network is trained a little further on the labelled reviews, with a small learning rate so that the learned knowledge is adjusted, not erased. In contrast to the BiLSTM, which learns language from the 315,000 training reviews alone, BERT starts with the knowledge of billions of words, and the project tests how much this helps. `bert-base-uncased` is the smaller of the two original sizes (12 layers, 109 million parameters) and lower-cases the text, which keeps the setup simple.

6.3 BiLSTM with Attention

Input

The input text of Section 5.3 is lower-cased and split by a regular expression that keeps contractions (*don't*) and the marks `!` and `?`. The vocabulary holds the words that occur at least three times in the training part, capped at 50,000; it ends with 46,760 entries, including the

padding and unknown symbols. It covers 99.71% of the training tokens. Each review is cut at 200 words, which cuts 6.47% of the training reviews. Batches of 256 reviews are formed by length bucketing: the data are shuffled, grouped into chunks, sorted by length inside each chunk and cut into batches, so that reviews of similar length share a batch and padding is small. There are 1,230 batches per epoch.

Architecture

Layer	Role
Embedding 46,760 × 128	a trainable vector per word, dropout 0.3
BiLSTM, 128 units per direction	reads the review left-to-right and right-to-left; the state at position t is $h_t \in \mathbb{R}^{256}$
Attention pooling	weights each position and sums the states
Dropout 0.3, Linear $256 \to 3$	class scores

The attention layer scores each state, normalises the scores over the real (non-padding) positions, and uses the weighted sum as the review vector:

$$e_t = \mathbf{w}^\top h_t, \qquad \alpha_t = \frac{\exp(e_t)}{\sum_k \exp(e_k)}, \qquad \mathbf{c} = \sum_t \alpha_t h_t.$$

Padding positions get a score of -10^9 before the softmax, so their weight is zero. The model has 6,250,500 trainable parameters, of which 5,985,280 are the embedding.

Training

Setting	Value
Optimiser	AdamW, learning rate 2×10^{-3}, weight decay 10^{-4}
Schedule	learning rate halved when validation Macro-F1 does not improve
Gradient clipping	norm 1.0
Epochs, early stopping	at most 5; stop after 2 epochs without improvement; keep the best epoch
Seed	42

The run used all 5 epochs; the best validation Macro-F1 is 0.7504 at epoch 4, and the model of that epoch is kept. Training takes 3.4 minutes on one Tesla T4 GPU.

Epoch	Train loss	Validation loss	Validation accuracy	Validation Macro-F1	Learning rate
1	0.6168	0.5283	0.8646	0.7275	0.002
2	0.4956	0.5292	0.8830	0.7469	0.002
3	0.4549	0.5049	0.8614	0.7318	0.002
4	0.4100	0.5376	0.8820	**0.7504**	0.001
5	0.3894	0.5342	0.8772	0.7469	0.001

The training loss falls steadily while the validation loss stays near 0.53 and is lowest at epoch 3, which signals overfitting from about epoch 3 or 4. The epoch is chosen by Macro-F1, not by validation loss, and the differences between the last epochs (about 0.003) are small.

6.4 Fine-tuning BERT

`bert-base-uncased` is a 12-layer Transformer encoder (hidden size 768, 12 attention heads) pre-trained on Wikipedia and BookCorpus. A new three-class layer is placed on the representation of the `[CLS]` token and the whole network is trained. The configuration is a single run without hyperparameter search.

Setting	Value
Tokenisation	WordPiece, cut at 256 tokens; dynamic padding with length bucketing
Truncation	about 6.2% of reviews (measured on 3,000 training reviews; median length 80, 90th percentile 207 tokens)
Batch, epochs	32 reviews, 2 epochs; 9,840 steps per epoch, 19,680 in total
Optimiser	AdamW, learning rate 3×10^{-5}, weight decay 0.01 (not applied to biases and LayerNorm)
Schedule	linear warm-up over the first 10% of steps, then linear decay to 0
Other	fp16 mixed precision, gradient clipping 1.0, seed 42
Parameters	109,484,547

After each epoch the model is scored on the validation part and saved if its Macro-F1 is the best so far; there is no early stopping. Each epoch takes 32.2 minutes on a Tesla T4, 64.4 minutes in total.

Epoch	Train loss	Validation accuracy	Validation Macro-F1
1	0.4943	0.8884	0.7664
2	0.3517	0.9069	**0.7864**

Validation Macro-F1 is still rising at the last epoch, so the model has not saturated and more epochs might help; with two epochs overfitting cannot be observed.

6.5 Summary of the Models

Model	Trained on this data	Parameters	Selection on validation
BiLSTM + attention	yes, from scratch	6.25 M	epoch, early stopping
BERT-base	yes, fine-tuned	109.48 M	epoch
Always positive	no	0	–

Both networks were trained on one Tesla T4 GPU, so their training times are comparable.

Chapter 7

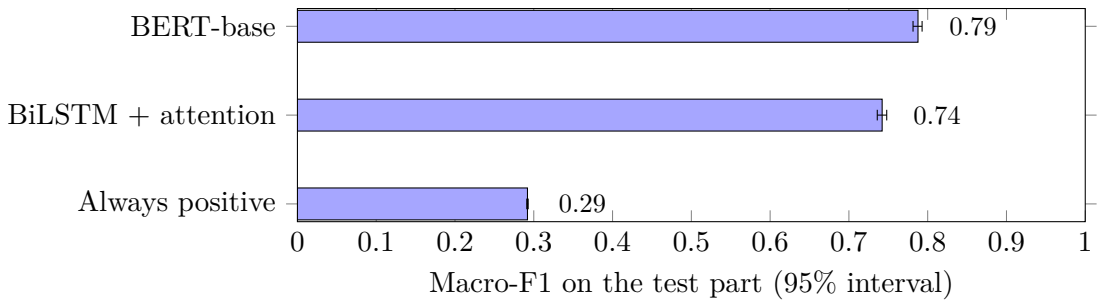
Evaluation

7.1 Objective

Measure how well each model assigns sentiment to reviews it has not seen, compare the two networks fairly, and find where they fail. The protocol is fixed before the results: Macro-F1 (Section 2.4) is the main metric; the test part of 39,358 reviews is read once; every choice (epoch, neutral threshold) is made on the validation part. Confidence intervals come from 1,000 bootstrap resamples of the test reviews, with all models scored on the same resample so that differences can be paired. The intervals reflect the sampling error of the test part only, not the choice of random seed.

7.2 Results on the Full Test Part

Model	Accuracy	Macro-F1	F1 negative	F1 neutral	F1 positive	Macro-F1 95% interval
BERT-base fine-tuned	**0.9074**	**0.7877**	**0.8470**	**0.5540**	**0.9620**	[0.7814, 0.7929]
BiLSTM + attention	0.8788	0.7422	0.7863	0.4916	0.9488	[0.7360, 0.7480]
Always positive	0.7794	0.2920	0	0	0.8760	[0.2911, 0.2929]



Both networks are far above the always-positive rule (Macro-F1 0.292, which scores 0.78 accuracy while recognising nothing about the other two classes). BERT is best on every column. The F1 of the neutral class is the lowest for both networks, 0.49 and 0.55.

Cost. Both networks were trained and timed on a Tesla T4.

Model	Training time	Inference on the test part	Parameters
BERT-base	64.4 min	83.7 s	109.48 M
BiLSTM + attention	3.4 min	3.7 s	6.25 M

BERT needs about 19 times the training time, 23 times the inference time and 17.5 times the parameters of the BiLSTM for 0.0455 more Macro-F1. The BiLSTM reaches about 94% of the Macro-F1 of BERT with about 5% of its training time.

7.3 Is the Neutral Class the Problem?

Both networks have high recall but low precision on neutral: they predict neutral for too many reviews. The table gives, for each model, the share of true neutral reviews that are predicted correctly, and the true label of the reviews it predicts as neutral.

	True neutral predicted as			Predicted neutral are truly		
Model	negative	neutral	positive	negative	neutral	positive
BERT-base	19.7%	68.1%	12.1%	17.9%	46.7%	35.4%
BiLSTM + attention	16.5%	67.6%	15.9%	23.4%	38.6%	38.0%

Both networks recover about two thirds of the neutral reviews, but fewer than half of the reviews they call neutral are neutral. The cause is partly the data (a 3-star review often praises and complains at once, and the neutral class has no vocabulary of its own, Section 4.8) and partly the loss, whose neutral weight is 4.4 times a typical weight (Section 5.6).

7.4 Performance by Review Length

Words per review	≤30	31–60	61–100	101–200	201–400	>400
Reviews in the test part	6,137	13,659	9,601	7,534	2,100	327
BERT-base	0.778	0.783	0.794	0.796	0.758	0.693
BiLSTM + attention	0.753	0.738	0.746	0.747	0.704	0.600

Both models are lowest on reviews longer than 400 words, and the BiLSTM drops the most (0.600 against 0.693 for BERT). The BiLSTM keeps only the first 200 words and BERT the first 256 tokens, so both lose the end of a long review. The last group has only 327 reviews, so its scores vary more and should not be over-read.

7.5 Bootstrap Interval for the Difference

The difference in Macro-F1 between BERT and the BiLSTM on the full test part is +0.0454, with a 95% interval of [+0.0400, +0.0507] over the paired bootstrap resamples. The interval does not contain zero, so the difference is larger than the sampling error of the test part, and BERT is ahead of the BiLSTM in 100% of the resamples.

7.6 Calibrating the Neutral Threshold

Because the models over-predict neutral, a constant `bias` is added to the log-probability of neutral before taking the arg max (a negative bias predicts neutral less often). The bias is chosen on the validation part to maximise Macro-F1 and then applied once to the test part. This needs no retraining.

Model	Chosen bias	Validation Macro-F1 before → after	Test Macro-F1 before → after	Test F1 neutral before → after
BiLSTM + attention	−0.8	0.7504 → 0.7577	0.7422 → 0.7551	0.4916 → 0.4978
BERT-base	−1.1	0.7864 → 0.7902	0.7877 → 0.7866	0.5540 → 0.5448

The adjustment helps the BiLSTM (+0.013 on the test part) but not BERT: the gain of 0.004 that BERT shows on validation does not carry over to the test part and lies within the noise. The headline results of this report use no bias (Section 7.2); the calibration is an extra experiment.

7.7 Error Analysis and Examples

Confident errors of the best model

BERT predicts *negative* for 165 reviews of 5 stars and *positive* for 58 reviews of 1 star; 948 neutral reviews are misclassified. Many confident errors look like noisy labels: the author gave 5 stars but wrote a complaint ("Now I would not recommend it to my worst enemy. The coffee now has no aroma…", "Incredibly overpriced…this is a total and complete rip off"), or 1 star with praise ("LOVE LOVE LOVE THESE! (edited because NO longer available in prime)"). In these cases the model reads the text better than the star count does. Not every case is noise: "My dogs won't eat anything else. My dogs will not eat anything but the Iams biscuit" praises the product, and the model misreads it. The share of noisy labels among the errors was not counted, so this report does not give a rate.

A related property of the data: about 3.4% of test reviews (1,350) contain a phrase such as "3 stars", and only about 59% of them state the true rating (the others read "5 star service" or "4 stars out of 5"). These texts are kept, so some signal flows from the label into the text for both models.

Sentences written for the test

Six sentences written by the authors give a qualitative check; they are not a substitute for the measurements above.

Sentence	BiLSTM	BERT
"This coffee is absolutely amazing, I will definitely buy it again!"	pos (100%)	pos (100%)
"Not bad at all, actually pretty tasty."	neu (78%)	pos (79%)
"It's okay. Nothing special, but it does the job."	neu (100%)	neu (99%)
"Great, another box arrived completely crushed. Just what I needed."	pos (87%)	pos (99%)
"My dog loves these treats, but they are way too expensive for such a small bag."	neu (84%)	neu (89%)
"I would not recommend this to anyone. Total waste of money."	neg (100%)	neg (100%)

Both models read the sarcastic fourth sentence as positive, because "great" and "just what I needed" sound positive. Both label the mixed third and fifth sentences neutral, and the clearly positive and negative sentences correctly. For the second sentence the BiLSTM answers neutral and BERT positive; both are acceptable for such mild praise. The attention weights of the BiLSTM mostly fall on words that carry sentiment or reverse it (*mistake*, *great*, *not*, *okay*, *but*), but not always: for one neutral review the largest weight is on a product name (*stevia*). Attention shows where the model looks, not a complete explanation.

7.8 Limitations

- **Weak labels.** Stars do not always match the text (Section 7.7), and the 3-star boundary is subjective, which bounds the quality any model can show.
- **Sarcasm and mixed opinions.** Both models fail on sarcasm, and neutral stays the hardest class (F1 0.49 and 0.55).
- **Random split.** The label distribution drifts over time (Section 4.5) and the same user or product can sit in both training and test, so the scores do not measure performance on future reviews.

- **Runs and seeds.** Each model was trained once (seed 42), so the bootstrap intervals do not cover variation between seeds.
- **BERT is not saturated.** It was fine-tuned for 2 epochs with one fixed configuration, so its score may be a lower bound.
- **Different truncation.** The BiLSTM keeps 200 words and BERT 256 tokens.
- **One domain.** All data are English food reviews from Amazon, 1999–2012.

Chapter 8

Conclusion

8.1 Summary

The project turned 568,454 Amazon food reviews into a leakage-safe corpus of 393,577 cleaned reviews (Chapter 5), split 80/10/10 with stratification, and compared two trained classifiers with the always-positive rule on a three-class sentiment task labelled from star ratings.

- **Fine-tuned BERT-base is the better model:** Macro-F1 0.7877 and accuracy 0.9074 on the 39,358 test reviews, ahead of the BiLSTM with attention (0.7422) by 0.0455, with a 95% interval of [+0.0400, +0.0507] for the difference. It is ahead on all three classes, so transfer learning beats learning from scratch on this task.
- **Neutral is the weak point of both models** (F1 0.49 and 0.55): they predict it too often, and shifting the threshold helps only the BiLSTM.
- **Cost.** BERT needs 19 times the training time and 17.5 times the parameters of the BiLSTM for 0.0455 more Macro-F1.

8.2 Limitations

The limits are listed with their measurements in Section 7.8. The most important are: star labels that do not always match the text; a random split that does not measure performance on future reviews; a single run for each model; and a BERT model that was fine-tuned for only two epochs, with validation Macro-F1 still rising.

8.3 Future Work

1. **Longer or repeated fine-tuning of BERT** (more epochs, several seeds), and stronger encoders such as RoBERTa-base or DeBERTa-v3, to test whether the BERT score is a lower bound.
2. **A time-based split** (train until 2011, test on 2012) to measure performance on later reviews and the effect of the drift of the label distribution.
3. **Pre-trained word vectors** (GloVe, fastText) for the BiLSTM embedding, to see whether the gap to BERT narrows.
4. **Other ways to treat the imbalance** (focal loss, oversampling), because the large neutral weight may cause the over-prediction of neutral.
5. **Head-and-tail truncation** for long reviews, where both models scored lowest.

6. **A small human-labelled sample**, not based on stars, to measure how much of the error is label noise.

8.4 Reproducibility

The split uses a fixed seed and rule and all predictions on the test part are stored, so every table of Chapter 7 can be recomputed from them. The code is one notebook (cleaning, understanding and preparation of the data, BiLSTM, BERT and evaluation) and a shared package `src/`.

Bibliography

[1] J. McAuley and J. Leskovec. From amateurs to connoisseurs: modeling the evolution of user expertise through online reviews. In *Proceedings of the 22nd International Conference on World Wide Web (WWW)*, 2013.

[2] Stanford Network Analysis Project. Amazon Fine Food Reviews (Kaggle dataset `snap/amazon-fine-food-reviews`). https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews.